

Real-Time Chat Server built in Months 4–6 using

FastAPI/WebSockets, Redis pub/sub, NGINX, Docker, and AWS. Its stated milestones are **rooms, presence, rate limiting, load testing with Locust, and blue/green deployment**, and its intended outcomes are **concurrency, state management, scale, and rollout strategies**.

The roadmap also gives the implementation sequence for Project 2:

- first build **WebSocket endpoints + room model**
- then **presence service + heartbeats**
- then **Redis pub/sub fan-out scaling**
- then **rate limiting (token bucket) + abuse prevention**
- then **load test with Locust and identify bottlenecks**.

1) What Project 2 actually is

This project is not just “make a chat app.” It is a backend-focused system-design project meant to teach you:

- real-time bidirectional communication using WebSockets
- multi-user room-based messaging
- distributed state coordination across server instances
- horizontal scaling using Redis pub/sub
- protection against abuse with rate limiting
- performance testing under concurrency
- production deployment practices using NGINX, Docker, AWS, and blue/green rollout.

So the real purpose is to learn how a chat backend behaves when there are many simultaneous users and possibly multiple backend instances.

2) What is explicitly required by the file

These are the things the PDF clearly requires:

Required stack

- FastAPI + WebSockets
- Redis pub/sub
- NGINX
- Docker
- AWS

Required feature milestones

- rooms
- presence
- rate limiting
- Locust load testing
- blue/green deployment

Required implementation progression

- WebSocket endpoints + room model
- presence service + heartbeats
- Redis fan-out scaling
- token-bucket rate limiting + abuse prevention
- load test and bottleneck analysis

3) What the file does not explicitly specify

The PDF does **not** explicitly define:

- exact REST endpoints
- exact WebSocket message schema
- authentication method
- whether messages must be persisted in a database
- whether private chat is needed
- whether read receipts, typing indicators, attachments, or moderation are required

So the endpoint plan below is a **recommended implementation spec** based on the stated milestones, not a verbatim requirement from the PDF.

Detailed Project 2 plan

4) Functional scope I recommend

To satisfy the roadmap properly, your Project 2 should support:

1. User connects to chat server
2. User joins a room
3. User sends messages to a room
4. Other room members receive messages in real time

5. Presence is tracked
6. Heartbeats mark users online/offline
7. Multiple FastAPI instances stay synchronized through Redis pub/sub
8. Abuse is limited with token-bucket rate limiting
9. System can be load-tested and deployed safely

That is the minimum strong version of this project.

5) Core architecture

Main components

- **FastAPI app**
 - REST endpoints for room/user metadata
 - WebSocket endpoint for live chat
- **Redis**
 - pub/sub for cross-instance message fan-out
 - presence cache / ephemeral state
 - rate-limit counters/token buckets
- **NGINX**
 - reverse proxy
 - WebSocket upgrade handling
 - optional load balancing across app containers
- **Docker / Docker Compose**
 - app container
 - redis container
 - nginx container
- **AWS**
 - deployment target
- **Locust**
 - simulate concurrent users/messages

Recommended internal flow

1. Client opens WebSocket connection
2. Server authenticates/identifies user
3. User joins room

4. Message from user is validated
 5. Message is published to Redis channel for that room
 6. All app instances subscribed to that room channel fan out to their connected clients
 7. Presence keys in Redis are refreshed by heartbeats
 8. Rate limiter blocks spammy users
-

Endpoints that should exist

6) REST endpoints

Even though this is a WebSocket-heavy system, you still need REST endpoints for setup, health, and metadata.

Health / ops

GET /health

Purpose: basic health check for app

GET /ready

Purpose: readiness check, confirms Redis connection

GET /metrics

Purpose: optional Prometheus-style metrics if you add observability later

Auth / identity

The PDF does not require auth explicitly, but practically you need some user identity. For a first version, keep it simple.

POST /auth/guest

Purpose: create temporary identity or token for chat session

Response:

- user_id
- username/display_name
- access_token or session token

You could also skip JWT at first and use a simple generated session token.

Rooms

POST /rooms

Purpose: create room

Body:

- name
- visibility (public / private optional)

GET /rooms

Purpose: list available rooms

GET /rooms/{room_id}

Purpose: room details

POST /rooms/{room_id}/members

Purpose: join room through HTTP if you want explicit membership registration

Body:

- user_id

DELETE /rooms/{room_id}/members/{user_id}

Purpose: leave room

GET /rooms/{room_id}/members

Purpose: list current members or currently online members

Presence

GET /presence/{room_id}

Purpose: list online users in a room

GET /presence/users/{user_id}

Purpose: get whether a specific user is online/offline/last_seen

Messages

If you choose message persistence:

GET /rooms/{room_id}/messages

Purpose: fetch recent message history

Query params:

- limit
- before
- after

The PDF does not explicitly require persistence, but this endpoint is useful and realistic.

7) WebSocket endpoint

Main real-time endpoint

WS /ws/chat

This should be your primary WebSocket endpoint.

Recommended connection query/header inputs:

- token or session id
- optional room_id if auto-joining on connect

WebSocket events/messages that must be supported

Client → Server

connect

initial auth/handshake if needed

join_room

```
{
  "type": "join_room",
  "room_id": "room_123"
}
```

leave_room

```
{
  "type": "leave_room",
  "room_id": "room_123"
}
```

send_message

```
{
  "type": "send_message",
  "room_id": "room_123",
  "client_message_id": "uuid",
  "content": "hello"
}
```

heartbeat

```
{
  "type": "heartbeat",
  "timestamp": 1710000000
}
```

Optional:

typing_start, typing_stop

Server → Client

connection_ack

```
{
  "type": "connection_ack",
  "user_id": "user_1"
}
```

room_joined

room_left

message

```
{  
  "type": "message",  
  "room_id": "room_123",  
  "message_id": "msg_456",  
  "sender_id": "user_1",  
  "content": "hello",  
  "sent_at": "2026-03-18T10:00:00Z"  
}
```

presence_update

```
{  
  "type": "presence_update",  
  "room_id": "room_123",  
  "user_id": "user_2",  
  "status": "online"  
}
```

rate_limited

```
{  
  "type": "rate_limited",  
  "retry_after_seconds": 10  
}
```

error

```
{  
  "type": "error",  
  "code": "ROOM_NOT_FOUND",  
  "message": "Room does not exist"  
}
```

Data / models you should have

8) Minimum domain models

User

- id
- display_name
- created_at

Room

- id
- name
- created_by
- created_at
- is_private

RoomMembership

- room_id
- user_id
- joined_at

Message

If persisting messages:

- id
- room_id
- sender_id
- content
- created_at
- client_message_id

Presence

Usually not stored permanently in SQL. Better in Redis:

- user_id
 - room_id
 - status
 - last_seen
 - connection_id
-

Implementation details you should follow

9) Room model

Because the roadmap explicitly says “**WebSocket endpoints + room model**”, your room model should support:

- many users per room
- a mapping of room → active WebSocket connections
- a separate persistent membership concept if needed
- broadcast only to connections subscribed to the room.

A typical in-memory structure per app instance:

- room_id -> set[connection]

- `connection_id` -> user/session info

But because in-memory state breaks when you scale to multiple instances, Redis pub/sub becomes necessary.

10) Presence service + heartbeats

The roadmap explicitly requires **presence service + heartbeats**.

Recommended logic

- When client connects, mark user online in Redis
- Every 20–30 seconds client sends heartbeat
- Server updates Redis TTL key such as:
 - `presence:user:{user_id}` with expiry
- If TTL expires, user is considered offline
- Join/leave room events also update room presence sets

Redis structures

- `presence:user:{user_id}` → last seen / connection metadata with TTL
- `presence:room:{room_id}` → set of online user ids
- `conn:{connection_id}` → user_id, room membership, expiry

Why heartbeats matter

WebSockets can disconnect silently. Heartbeats help you clean stale online states.

11) Redis pub/sub fan-out scaling

The roadmap explicitly requires **Redis pub/sub fan-out scaling**.

This means:

- if you run 3 FastAPI instances behind NGINX
- user A may be connected to instance 1
- user B may be connected to instance 3
- when A sends a room message, all instances must know to deliver it to their local connections

Recommended flow

1. User sends `send_message`
2. Server validates it
3. Server publishes to Redis channel:

- `room:{room_id}`
4. Every app instance subscribed to that channel receives the event
 5. Each instance broadcasts to its own local WebSocket clients in that room

This is the key scaling feature of the project.

12) Rate limiting and abuse prevention

The roadmap explicitly requires **token bucket rate limiting + abuse prevention**.

What to rate-limit

At minimum:

- messages per user per minute
- room joins per minute
- new connections per IP/session per minute
- invalid event spam

Recommended token buckets

- WebSocket connect bucket
- message send bucket
- room action bucket

Example rules

- max 20 messages/minute/user/room
- max 5 room joins/minute/user
- max 10 connection attempts/minute/IP

When limited

Do not instantly close connection unless abuse is severe. Usually:

- send `rate_limited`
 - optionally mute briefly
 - close only repeated abusers
-

Non-functional requirements

13) Requirements your system should meet

Performance

- low-latency room broadcast
- stable under many concurrent connections
- ability to handle burst traffic

Scalability

- more than one app instance
- Redis-based coordination for distributed delivery

Reliability

- clients should reconnect gracefully
- stale presence should expire automatically
- invalid room/message events should not crash connections

Security

- validate all payloads
- sanitize message size
- restrict room access if private
- do not trust client heartbeat blindly

Deployment

- Dockerized services
 - NGINX reverse proxy with WebSocket upgrade support
 - AWS deployment
 - blue/green rollout support.
-

Concrete build phases

14) Best implementation phases

Phase 1 — skeleton

- FastAPI app

- Redis connection
- Docker Compose
- health endpoint
- simple WebSocket connect/disconnect logging

Phase 2 — rooms

- create/list rooms
- join/leave room
- room-scoped broadcast
- room connection manager

Phase 3 — presence

- online/offline state
- heartbeats
- last_seen
- presence updates to room members

Phase 4 — scale-out

- Redis pub/sub per room
- subscribe/unsubscribe logic
- multi-instance testing locally

Phase 5 — protection

- token bucket rate limiter in Redis
- block abusive send frequency
- message size limits

Phase 6 — production

- NGINX reverse proxy
- multiple app replicas
- AWS deployment
- blue/green strategy

Phase 7 — validation

- Locust load tests
- measure latency, disconnects, dropped messages, throughput

- identify bottlenecks
-

Suggested folder structure

15) Backend structure

```
app/  
  main.py  
  core/  
    config.py  
    security.py  
    rate_limit.py  
  api/  
    routes/  
      health.py  
      auth.py  
      rooms.py  
      presence.py  
      messages.py  
  ws/  
    handlers.py  
    manager.py  
    events.py  
    schemas.py  
  services/  
    room_service.py  
    presence_service.py  
    pubsub_service.py  
    message_service.py  
  redis/  
    client.py  
    channels.py  
  models/  
    user.py  
    room.py  
    membership.py  
    message.py  
  schemas/  
    room.py  
    message.py  
    presence.py  
tests/  
docker/  
nginx/  
locust/
```

Recommended minimum deliverable

16) What your final Project 2 should definitely demonstrate

At minimum, by the end, someone should be able to:

1. create a room

2. open two or more clients
3. join the same room
4. send messages and see real-time delivery
5. see online/offline presence updates
6. run multiple backend instances and still receive messages correctly through Redis
7. trigger rate limits with spam
8. run Locust and observe throughput/latency
9. deploy via Docker + NGINX on AWS
10. explain how blue/green deployment would be done

That would match the roadmap well.

Final distilled spec

17) Project 2 spec, in one paragraph

Project 2 is a backend-heavy **real-time room-based chat server** whose main goal is to teach **WebSocket concurrency, room state management, distributed fan-out via Redis pub/sub, online presence tracking through heartbeats, abuse protection through token-bucket rate limiting, performance evaluation with Locust, and production deployment using NGINX, Docker, AWS, and blue/green rollout strategy**. The PDF explicitly requires the stack and milestones, while the exact API contract is left open, so the endpoint design above is the practical implementation spec I recommend.